



MCDI500 · Magíster en Ciencia de Datos e Inteligencia Artificial

Guía de Git y GitHub

Documento técnico de apoyo · Programación para la Ciencia de Datos Control de versiones y reproducibilidad en el ecosistema científico de Python

Versión para macOS · Facultad de Ingeniería · Universidad Andrés Bello · 2026

Contenidos

	Parte	Pág.
	Cómo usar esta guía — ruta de lectura, convenciones y mapa con la rúbrica	
I	Control de versiones y reproducibilidad en investigación computacional	
II	El ecosistema científico: cuatro piezas que trabajan juntas	
III	Instalación y preparación del entorno	
IV	Crear el proyecto y su entorno	
V	Verificación del ecosistema completo	
VI	Los tres estados de Git y el primer flujo	
VII	El archivo <code>.gitignore</code>	
VIII	Notebooks y Git: higiene para ciencia de datos	
IX	Reproducibilidad del entorno	
X	Un día de trabajo completo, de principio a fin	
XI	GitHub: cuenta y repositorio del grupo	
XII	Conectar Git local con GitHub	
XIII	Flujo de trabajo en equipo	
XIV	Ramas	
XV	Colaboración con Pull Requests	
XVI	Conflictos: qué son y cómo resolverlos	
XVII	El README técnico	

	Parte	Pág.
XVIII	Mensajes de commit	
XIX	Diagnóstico de errores por acoplamiento	
XX	Integridad académica en el repositorio	
XXI	Checklist antes de cada entrega	
XXII	Secuencia consolidada desde cero	
	Glosario · Autoevaluación · Conclusión · Referencias	

Cómo usar esta guía

Tiempo de lectura | 110 minutos en total. No está pensada para leerse de una sola vez: consulte cada parte cuando la necesite, según la ruta de la sección siguiente.

Este documento acompaña las cuatro fases del proyecto transversal del módulo. No es un manual de referencia para consulta salteada: está ordenado como una secuencia argumentativa, y cada parte supone resueltas las anteriores.

El objetivo no es que usted memorice comandos. Es que comprenda **qué garantiza cada herramienta y bajo qué condiciones deja de garantizarlo**, porque esa es la diferencia entre un proyecto que otra persona puede auditar y uno que solo funciona en el computador donde se escribió.

Trabaje con el documento abierto junto a la Terminal. Ejecute los comandos, pero léalos antes: un repositorio construido a ciegas es identificable en la revisión —el historial lo evidencia— y no ofrece dónde apoyarse cuando algo falle en la Fase 3.

Se asume un punto de partida sin Python ni Git instalados. No se asume experiencia previa con la línea de comandos.

Nota — ¿usa Windows? Existe una versión equivalente de esta guía para Windows 10 y 11. Los conceptos y los comandos de Git son idénticos; cambian la instalación, la terminal y la activación del entorno virtual. Use la versión que corresponda a su equipo.

Convenciones

Recuadro	Significado
Nota	Información complementaria que aclara el concepto.
Atención	Punto donde se producen la mayoría de los errores. Léalo antes de ejecutar.
Recomendación	Práctica profesional que conviene adoptar desde ya.
Rúbrica	Vínculo explícito con un criterio evaluado del curso.
Comprueba	Verificación de cierre de cada parte: si no puede responderla, vuelva atrás.

Ruta de lectura por fase y semana

El módulo dura tres semanas y usted no necesita la guía completa desde el primer día.

Momento del curso	Partes	Para qué le sirve
Fase 0 — Diagnóstico	I, II	Comprender qué problema resuelve el control de versiones y cómo encajan las cuatro piezas.
Semana 1 · Fase 1	III a VII, XI, XII, XVII	Instalar el entorno, crear el repositorio del grupo, definir la estructura y el README.
Semana 1 · Fase 2	VIII, IX, X, XIII, XVIII	Higiene de notebooks, reproducibilidad, flujo en equipo y mensajes de commit.
Semana 2 · Fase 3	XIV, XV, XVI	Ramas, Pull Requests y resolución de conflictos.
Semana 3 · Fase 4	XVII, XXI	README definitivo, trazabilidad y checklist de entrega.
En cualquier momento	XIX, XX, glosario	Diagnóstico de errores, integridad académica y vocabulario.

Recomendación — antes de la Formativa 1. La Formativa 1 le pide representar en un mapa conceptual el flujo de trabajo reproducible del proyecto. Las Partes **II, V, VII y IX** describen exactamente ese flujo: cómo se articulan las herramientas, la estructura de carpetas, qué se versiona y qué no, y cómo se garantiza que otra persona pueda ejecutar el proyecto. Léalas antes de elaborar el mapa y tendrá la mitad del trabajo hecho.

Si usted ya trabaja con Git y GitHub

El grupo que ingresa al Magíster llega con experiencias muy distintas. Si usted ya usa Git con soltura, verifique que domina estos seis puntos y salte a las partes indicadas.

¿Sabe hacerlo?	Si no	Si sí
Explicar por qué el kernel del notebook y el entorno virtual deben coincidir	Parte II	Esta es la brecha más común, incluso con experiencia
Explicar la diferencia entre <code>git pull</code> y <code>git fetch</code>	Parte XIII	—
Agregar colaboradores y trabajar los cuatro sobre el mismo repositorio	Parte XI	—
Evitar que los notebooks generen conflictos ilegibles	Parte VIII	—
Resolver un conflicto de <i>merge</i> sin entrar en pánico	Parte XVI	—
Escribir un README que permita a un tercero ejecutar el proyecto	Parte XVII	—

En todos los casos, revise la **Parte XXI** (*checklist* de entrega) antes de cada evaluación: contiene los requisitos específicos de este curso, que no dependen de su experiencia previa.

Dónde pedir ayuda

- **Foro de Consultas** — dudas sobre contenidos y comandos. Antes de preguntar, indique qué comando ejecutó, qué esperaba y qué mensaje exacto recibió: así la respuesta llega en la primera iteración.
- **Foro de Tutoría** — problemas de acceso, plataforma o soporte técnico.
- **Cápsula 1: Implementa tu entorno reproducible** y el video de Jupyter de la Semana 1 — cubren en formato audiovisual lo mismo que las Partes III a VI.

Por qué esto se evalúa: mapa guía ↔ rúbrica

Git y GitHub no son un requisito administrativo. Son la **evidencia de cómo trabajó usted**. En ciencia de datos el resultado final no basta: se exige poder demostrar cómo se llegó a él, quién hizo qué y que otra persona pueda reproducirlo.

Criterio de la rúbrica	Qué mira el revisor	Partes
Control de versiones	Historial verificable, commits descriptivos y frecuentes, ramas cuando corresponde	VI, VII, XIV, XVIII
Repositorio GitHub	Carpetas por fase, README estructurado, commits identificables por integrante	XI, XII, XIII, XVII
Reproducibilidad técnica	Ambiente virtual, dependencias verificadas, ejecución sin errores en cualquier equipo	IV, V, IX
Documentación del proceso	Decisiones registradas, flujo explicado, cambios justificados con motivo e impacto	XVII, XVIII
Notebook ejecutado y documentado	Notebook que corre de principio a fin, con trazabilidad hacia el repositorio	VIII, IX

Rúbrica. El criterio de **Repositorio GitHub** exige contribuciones individuales identificables por integrante. Un repositorio donde todos los commits los realizó una sola persona no alcanza el nivel Excelente, aunque el contenido sea impecable. Esto no se corrige la noche anterior: el historial refleja cuándo trabajó cada quien.

Atención — equipos mixtos. Es habitual que en un grupo de cuatro convivan macOS y Windows. El repositorio, los comandos de Git y el `requirements.txt` funcionan igual en ambos; lo que cambia es la ruta de activación del entorno virtual (`.venv/bin/activate` en macOS, `.venv\Scripts\activate` en Windows). Documenten ambas en el README.

PARTE I. Control de versiones y reproducibilidad en investigación computacional

Tiempo de lectura | 8 min · Fase 0

Al terminar esta parte podrá fundamentar por qué el control de versiones es una exigencia metodológica y no una convención de la industria del software.

1.1 El problema epistémico

Un resultado computacional no se sostiene por sí mismo. Se sostiene si es posible **reconstruir el procedimiento que lo produjo**: qué datos entraron, qué transformaciones se aplicaron, en qué orden y bajo qué supuestos. Sin esa reconstrucción, un análisis es una afirmación sin respaldo verificable.

La literatura sobre investigación computacional distingue tres niveles, y conviene no confundirlos:

Nivel	Qué exige	Qué lo garantiza
Repetibilidad	El mismo equipo, con los mismos datos y código, obtiene el mismo resultado	Semillas fijadas, orden de ejecución determinista
Reproducibilidad	Otro equipo, con los mismos datos y código, obtiene el mismo resultado	Entorno declarado, rutas relativas, dependencias con versión
Replicabilidad	Otro equipo, con datos nuevos y método equivalente, obtiene conclusiones compatibles	Documentación del método, no solo del código

Este módulo evalúa los dos primeros. El tercero excede su alcance, pero conviene saber que existe: un trabajo puede ser perfectamente reproducible y no replicable, y eso también dice algo sobre el análisis.

Nota. Un proyecto puede ser **reproducible sin ser trazable** —corre igual, pero nadie sabe por qué se tomó cada decisión— o estar bien documentado sin ser reproducible. La rúbrica del módulo evalúa ambas dimensiones por separado, y por eso conviene distinguirlas desde el inicio.

1.2 La manifestación cotidiana del problema

La versión doméstica de todo lo anterior es reconocible:

```

 analisis_final.ipynb
 analisis_final_v2.ipynb
 analisis_final_v2_CORREGIDO.ipynb
 analisis_final_ESTESI.ipynb
 analisis_final_ESTESI_camila.ipynb

```

Esa carpeta no permite responder tres preguntas: qué cambió entre una versión y otra, **por qué** se hizo ese cambio, y cuál era el estado del proyecto el día en que los resultados eran los esperados. En trabajo colaborativo el problema se agrava, porque además ninguna persona sabe qué versión tiene el resto.

Nótese que el problema no es el desorden. Es que **la información sobre el proceso no existe en ninguna parte**: no se perdió, nunca se registró.

1.3 Git como sistema de registro

Git es un **sistema de control de versiones distribuido**: registra los estados sucesivos de un proyecto junto con su autoría, su momento y su justificación declarada.

«Distribuido» significa que cada copia del repositorio contiene el historial completo, no un puntero a un servidor central. Esa decisión de diseño tiene una consecuencia práctica relevante: usted puede trabajar, consultar el historial y registrar cambios sin conexión, y sincronizar después.

En ciencia de datos la utilidad es directa. Un experimento que ayer producía un resultado y hoy produce otro se explica, casi siempre, examinando qué cambió en el historial: una transformación distinta, un filtro agregado, una semilla modificada.

1.4 GitHub como infraestructura de colaboración

GitHub es una plataforma que aloja repositorios Git y añade sobre ellos una capa de colaboración: control de acceso, revisión de código mediante *Pull Requests*, seguimiento de incidencias y trazabilidad de la discusión técnica.

Nota. La distinción es funcional, no de marca. **Git** es el sistema de control de versiones y opera localmente. **GitHub** es un servicio de alojamiento y colaboración construido sobre Git. Existen alternativas equivalentes —GitLab, Bitbucket, Codeberg— y el conocimiento es transferible entre ellas. Se puede usar Git sin ninguna de las tres; no al revés.

Atención — error conceptual frecuente. Tratar GitHub como un servicio de almacenamiento en la nube. Si el repositorio se usa solo para depositar el archivo final, contendrá uno o dos commits de gran tamaño y ninguna historia. La rúbrica evalúa el historial como evidencia del proceso, no la existencia del archivo como producto.

Comprueba. ¿Podría fundamentar, ante alguien que sostenga que renombrar archivos con `_v2` cumple la misma función, por qué eso no constituye control de versiones? La respuesta no es «es más ordenado».

PARTE II. El ecosistema científico: cuatro piezas que trabajan juntas

Tiempo de lectura | 10 min · Fase 0

Al terminar esta parte entenderá qué hace cada herramienta, qué no hace, y dónde se conectan entre sí.

La mayoría de los problemas que enfrentará este semestre no ocurren *dentro* de una herramienta, sino **en el punto donde dos de ellas se tocan**. Por eso conviene ver el conjunto antes de instalar nada.

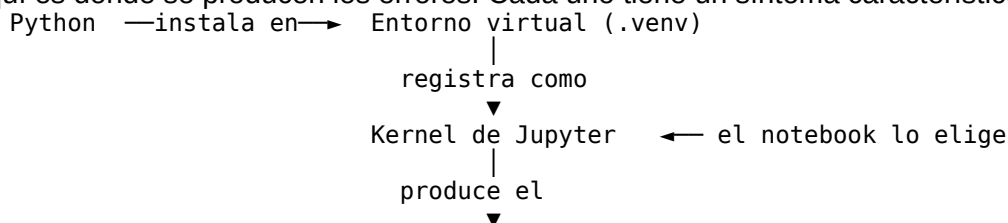
2.1 Qué hace cada pieza

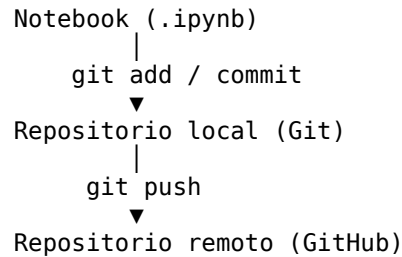
Pieza	Qué hace	Qué no hace
Python	Ejecuta el código y provee las librerías del análisis	No registra cambios ni documenta decisiones
Entorno virtual	Aísla las versiones de las librerías de este proyecto	No guarda su código ni lo comparte
JupyterLab	Integra código, resultados y narrativa en un documento	No versiona: guardar un notebook no es registrar un cambio
Git	Registra la historia del proyecto en su computador	No respalda en internet ni permite colaborar por sí solo
GitHub	Aloja el repositorio, habilita colaboración y revisión	No ejecuta su código ni verifica que funcione

Ninguna reemplaza a otra. Un notebook impecable sin control de versiones no es trazable; un historial perfecto sin entorno declarado no es reproducible.

2.2 Los cuatro puntos de acoplamiento

Aquí es donde se producen los errores. Cada uno tiene un síntoma característico.





#	Acoplamiento	Qué lo une	Síntoma cuando falla
1	Entorno ↔ Python	source .venv/bin/activate	Instala un paquete y «desaparece»; ModuleNotFoundError
2	Entorno ↔ Jupyter	ipykernel install y elegir el kernel	El notebook no encuentra pandas aunque esté instalado
3	Entorno ↔ reproducibilidad	requirements.txt	«A mí me funciona»: el compañero no puede ejecutar
4	Notebook ↔ Git	nbstripout	Diffs ilegibles y conflictos constantes
5	Git ↔ GitHub	git remote y autenticación	push rechazado, o el docente no ve el repositorio

Recomendación. Cuando algo falle, no busque el error en la herramienta donde apareció: pregúntese **qué acoplamiento se rompió**. La Parte XIX está organizada exactamente así.

2.3 El acoplamiento que más problemas causa

El número 2 merece detenerse. Un notebook no ejecuta el código «con Python» a secas: lo ejecuta con un **kernel**, que es un intérprete concreto. Si usted instaló pandas dentro de `.venv` pero el notebook está usando el kernel genérico «Python 3» del sistema, pandas no existe para ese notebook.

Es la causa número uno del clásico «a mí me funciona y a ti no»: cada integrante ejecutando contra un intérprete distinto.

```

# Esta celda, ejecutada dentro del notebook, dice EXACTAMENTE qué intérprete
# está usando. Si la ruta no contiene .venv, el kernel es el equivocado.
import sys
print(sys.executable)
  
```

Comprueba. Sin mirar la guía: ¿por qué instalar una librería con el entorno desactivado no soluciona un `ModuleNotFoundError` dentro del notebook?

PARTE III. Instalación y preparación del entorno

Tiempo de lectura | 10 min · Semana 1, Fase 1

Al terminar esta parte tendrá Git y Python funcionando, y Git sabrá quién es usted.

En macOS todo el trabajo se hace desde la aplicación **Terminal** (Aplicaciones → Utilidades → Terminal, o Spotlight con `⌘` + espacio escribiendo «Terminal»).

Nota. Es posible que haya visto guías que hablan de **Git Bash**: esa es la terminal que acompaña a Git en Windows. En macOS no existe Git Bash ni instaladores `.exe`; se usa la Terminal del sistema, con los mismos comandos de Git.

3.1 Instalar las herramientas de línea de comandos de Xcode

Incluyen un compilador y dependencias que las librerías científicas necesitan.

```
xcode-select --install
```

Acepte la ventana que aparece. Si ya están instaladas, el sistema se lo indicará.

3.2 Instalar Git

Opción A — Command Line Tools. El paso anterior normalmente ya instala Git. Verifíquelo:

```
git --version
```

Si Git no estuviera instalado, macOS le ofrecerá instalarlo; acepte.

Opción B — Homebrew. Si usted usa Homebrew, el gestor de paquetes de macOS:

```
brew install git
```

Nota. `git --version` debería mostrar algo similar a `git version 2.x.x`. El número exacto no importa: basta con que el comando responda.

3.3 Instalar Python 3

Descargue Python desde el sitio oficial: <https://www.python.org/>.

```
python3 --version
```

Atención. En macOS el comando `python` a secas no existe, y `command not found` es el primer tropiezo de casi todos. Mientras no active un entorno virtual, use siempre `python3` y `pip3`.

Recomendación — qué versión instalar. No instale la versión más reciente solo por ser la más nueva: las librerías científicas tardan meses en dar soporte completo a cada versión de Python. Verifique en la documentación de las librerías que usará —pandas, scikit-learn— qué versiones admiten, y elija una que todas soporten. Evite versiones anteriores a 3.11.

Atención — Mac con chip Apple (M1 a M4). Para que PyTorch use la GPU (*backend mps*) necesita macOS 12.3 o superior y un Python compilado de forma nativa para ARM64, no la versión emulada mediante Rosetta. Para trabajar con pandas y scikit-learn en este módulo no necesita nada de esto.

3.4 Configurar Git por primera vez

Git necesita saber quién realiza los commits.

```
git config --global user.name "Nombre Apellido"
```

```
git config --global user.email "correo@ejemplo.com"
```

```
git config --global init.defaultBranch main
```

```
git config --global --list
```

Atención. Use **el mismo correo** con el que creará su cuenta de GitHub. Si no coinciden, sus commits aparecerán en el historial sin vincularse a su perfil, y en la revisión no será posible atribuirle su trabajo. Corregirlo después implica reescribir el historial, que es un procedimiento delicado.

Nota. `--global` significa que la configuración aplica a todos los repositorios de su usuario en ese equipo. Definir `init.defaultBranch main` evita renombrar la rama más adelante.

Comprueba. Ejecute `git config --global user.email`. ¿Es el correo que usará en GitHub?

PARTE IV. Crear el proyecto y su entorno

Tiempo de lectura | 12 min · Semana 1, Fase 1

Al terminar esta parte tendrá la carpeta del proyecto con un entorno virtual propio y JupyterLab funcionando dentro de él.

Atención — el orden importa. Primero se crea la carpeta del proyecto, dentro de ella el entorno virtual, y recién entonces se instalan los paquetes. Así nunca se toca el Python del sistema —que en macOS responde con el error `externally-managed-environment`— y cada proyecto queda aislado con sus propias versiones.

4.1 Crear la carpeta del proyecto

```
mkdir proyecto-grupoX-mcdi500
cd proyecto-grupoX-mcdi500
```

4.2 Crear y activar un entorno virtual

```
python3 -m venv .venv
source .venv/bin/activate
```

Cuando el entorno está activo verá (.venv) al inicio de la línea. A partir de ese momento sí puede usar python y pip a secas, porque apuntan al entorno.

Nota. Para salir del entorno: deactivate. Recuerde volver a ejecutar source

.venv/bin/activate **cada vez que abra una terminal nueva**. Si instala un paquete y «desaparece», casi siempre es porque el entorno no estaba activo.

Qué es realmente .venv. Es una carpeta con una copia del intérprete y sus propias librerías. Activarla no ejecuta nada permanente: solo modifica, en esa terminal, a qué Python apuntan los comandos. Por eso hay que repetirlo en cada terminal nueva, y por eso .venv/ va al .gitignore: se reconstruye desde el requirements.txt.

4.3 Instalar la base de trabajo

```
python -m pip install --upgrade pip
python -m pip install jupyterlab notebook ipykernel \
    numpy pandas matplotlib seaborn scikit-learn
```

Abra JupyterLab con:

```
python -m jupyter lab
```

4.4 Registrar el kernel del proyecto

Este es el paso que resuelve el acoplamiento 2 de la Parte II.

La terminal anterior quedó ocupada ejecutando JupyterLab, así que abra una **segunda terminal** (⌘ + N), sitúese de nuevo en la carpeta del proyecto con cd, active el entorno y ejecute:

```
source .venv/bin/activate
python -m ipykernel install --user --name grupoX_mcdi500 \
    --display-name "Python (grupoX-mcdi500)"
```

--name es el identificador interno; --display-name es lo que verá en JupyterLab.

Atención. Al crear un notebook, elija el kernel de su proyecto, **no** el genérico «Python 3». Si ya creó el notebook con el kernel equivocado: menú *Kernel* → *Change Kernel*.

4.5 Extensiones e integración con Git

```
python -m pip install jupyterlab-git jupyter nbstripout
```

- **jupyterlab-git** añade un panel de Git dentro de JupyterLab.
- **jupyter** empareja un notebook .ipynb con un .py, lo que facilita versionar y revisar el código.
- **nbstripout** limpia las salidas de los notebooks antes de cada commit (Parte VIII).

4.6 Inicializar Git en el proyecto

```
git init
```

Esto crea un repositorio Git local en la carpeta. Desde este momento Git puede rastrear los cambios.

4.7 Estructura de carpetas del proyecto del curso

El proyecto transversal avanza por fases y el repositorio debe reflejar esa progresión.

```
proyecto-grupoX-mcdi500/
```

```
├── F1/
│   ├── data/raw/           datos originales, sin modificar
│   └── data/processed/     datos tras limpieza y transformación
```

```

├─ notebooks/F1_Definicion.ipynb
├─ src/                  funciones reutilizables (.py)
├─ docs/                 documentación y referencias
├─ F2/
├─ F3/
├─ F4/
├─ requirements.txt
├─ .gitignore
└─ README.md

```

Rúbrica. El criterio de **Repositorio GitHub** en la Sumativa 1 pide explícitamente carpetas F1 y F2, README estructurado e historial completo. No improvisen la estructura: definirla en la Fase 1 y respetarla evita reorganizaciones masivas que ensucian el historial.

Recomendación. Nunca modifique los archivos de data/raw/. Los datos originales son la referencia contra la cual se verifica todo lo demás; si los sobrescribe, pierde la posibilidad de demostrar qué transformó.

4.8 Crear los archivos iniciales

```
touch README.md
```

Para el notebook, en JupyterLab: desde el *Launcher*, sección **Notebook**, o desde *File* → *New* → *Notebook*. Elija el kernel del proyecto, renómbrelo y guárdelo en F1/notebooks/.

El .gitignore lo crearemos con contenido en la Parte VII, antes del primer commit.

PARTE V. Verificación del ecosistema completo

Tiempo de lectura | 6 min · Semana 1, Fase 1

Al terminar esta parte sabrá, con evidencia, que las cuatro piezas están conectadas.

Antes de escribir una línea de análisis conviene comprobar los cinco acoplamientos. Hacerlo ahora ahorra horas después.

5.1 Desde la Terminal

```
# 1. ¿El entorno está activo? Debe verse (.venv) al inicio de la línea
which python          # debe apuntar a ../proyecto-grupoX-mcdi500/.venv/bin/python
```

```
# 2. ¿Git conoce el proyecto y sabe quién es usted?
git status
git config user.email
```

```
# 3. ¿El kernel quedó registrado?
python -m jupyter kernelspec list    # debe aparecer grupoX_mcdi500
```

5.2 Desde el notebook

Cree una celda en su notebook y ejecute esto. Es la comprobación que resuelve la mayoría de los «no me funciona».

```
import sys
from pathlib import Path
```

```
# El intérprete que ejecuta ESTE notebook. Si la ruta no contiene .venv,
# el kernel es el equivocado: Kernel -> Change Kernel.
print("Intérprete:", sys.executable)
```

```
# La carpeta desde la que se ejecuta. Las rutas relativas del notebook
# se resuelven desde aquí, no desde donde está el archivo .ipynb.
print("Carpeta de trabajo:", Path.cwd())
```

```
# Las librerías del proyecto, con su versión
```

```
for lib in ["numpy", "pandas", "matplotlib", "sklearn"]:
    try:
        print(f" {lib:12}", __import__(lib).__version__)
    except ImportError:
        print(f" {lib:12} NO DISPONIBLE en este kernel")
```

5.3 Cómo interpretar el resultado

Lo que ve	Qué significa	Qué hacer
La ruta del intérprete no contiene <code>.venv</code>	Kernel equivocado (acoplamiento 2)	<i>Kernel</i> → <i>Change Kernel</i>
Una librería aparece como no disponible	Se instaló fuera del entorno (acoplamiento 1)	Active el entorno y reinstale
La carpeta de trabajo no es la raíz del proyecto	Sus rutas relativas fallarán	Abra JupyterLab desde la raíz
<code>git status</code> responde <i>not a git repository</i>	Falta <code>git init</code>	Ejécute en la raíz del proyecto

Recomendación. Deje esta celda como primera celda de todos sus notebooks. Cuesta tres segundos ejecutarla y responde de inmediato la pregunta «¿el problema es mi código o mi entorno?».

Comprueba. ¿Ve `.venv` en su terminal? ¿El notebook muestra «Python (grupoX-mcdi500)» como kernel en la esquina superior derecha? ¿`sys.executable` apunta dentro de `.venv`?

PARTE VI. Los tres estados de Git y el primer flujo

Tiempo de lectura | 8 min · Semana 1, Fase 1

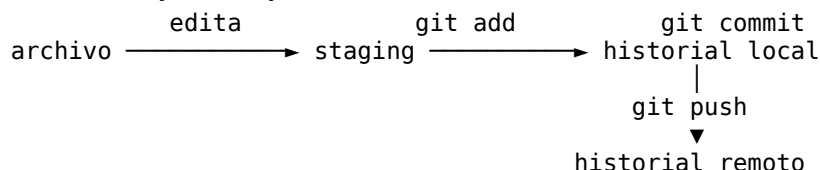
Al terminar esta parte entenderá por dónde pasa un archivo antes de quedar guardado en el historial, y habrá realizado su primer commit.

6.1 Los tres estados

Working Directory (directorio de trabajo). Donde usted crea, edita y guarda archivos. Git detecta los cambios pero todavía no los ha registrado.

Staging Area (zona de preparación). Zona intermedia donde se colocan los cambios que se quieren incluir en el próximo commit. Existe para poder **elegir qué entra**: si modificó tres archivos y solo dos corresponden al cambio que desea registrar, prepara esos dos.

Committed (confirmado en el historial). Git guardó esa versión de forma permanente, con autor, fecha y mensaje.



Nota. Un **commit** es un registro guardado de cambios: una instantánea del proyecto en un momento concreto, con autoría, fecha y mensaje.

El modelo de datos, en breve

Conviene saber qué hace Git por debajo, porque explica varios comportamientos que de otro modo parecen arbitrarios.

Git **no guarda diferencias entre versiones**: guarda instantáneas completas del árbol de archivos, y reutiliza internamente el contenido que no cambió. Cada commit queda identificado por un *hash* criptográfico —la cadena de cuarenta caracteres que aparece en `git log`— calculado sobre su contenido, su autoría y **el identificador del commit anterior**.

De ahí se siguen tres consecuencias:

- El historial forma un **grafo acíclico dirigido**: cada commit apunta a su predecesor, y una fusión apunta a dos. Eso es lo que `git log --graph` dibuja.
- El historial es **verificable**: alterar un commit antiguo cambia su hash y, en cascada, el de todos los posteriores. Por eso reescribir la historia de una rama compartida es una operación delicada.
- Una rama no es una copia de los archivos, sino **un puntero móvil a un commit**. Crear una rama es una operación instantánea, y esa es la razón por la que conviene usarlas con soltura.

Atención. Lo anterior fundamenta la advertencia sobre `git push --force`: al forzar, usted reemplaza la cadena de commits del remoto por la suya. Los commits que otros hicieron sobre la cadena anterior dejan de ser alcanzables.

6.2 El primer flujo completo

Revisar el estado. Es el comando que más utilizará:

```
git status
```

Ejecútelo antes y después de cada operación mientras esté aprendiendo: es la forma de que Git deje de parecer una caja negra.

Preparar los cambios:

```
git add nombre_archivo    # un archivo específico
git add .                  # todo lo que cambió
```

Confirmar:

```
git commit -m "docs: crea estructura inicial del proyecto"
```

Ver el historial:

```
git log
git log --oneline
git log --oneline --graph --all    # útil cuando haya ramas
```

Recomendación. `git add .` es cómodo pero peligroso si no revisó antes con `git status`. Es la vía habitual por la que se sube por accidente la carpeta `.venv/` completa o un archivo de datos de 500 MB.

Comprueba. Ejecute `git log --oneline`. ¿Aparece su commit con su nombre? Verifique el autor con `git log -1 --format='%an <%ae>'`.

PARTE VII. El archivo .gitignore

Tiempo de lectura | 5 min · Semana 1, Fase 1

Al terminar esta parte su repositorio ignorará lo que no debe versionarse.

El `.gitignore` indica qué elementos Git no debe rastrear: entornos, caché, datos pesados y archivos sensibles.

La lógica detrás. Quedan fuera tres cosas: lo que se **regenera** (el entorno virtual, los datos procesados), lo que es **local** de cada equipo (cachés, archivos del sistema) y lo que es **secreto** (credenciales, tokens).

```
# Entorno virtual: se reconstruye con requirements.txt
.venv/
```

```
# Caché de Python
__pycache__/
*.pyc

# Jupyter
.ipynb_checkpoints/

# macOS
.DS_Store
.AppleDouble

# Variables de entorno y secretos
.env

# Datos pesados y modelos
*.ckpt
*.pth
```

Atención. Cree y complete el `.gitignore` **antes** de su primer `git add ..` Si no, corre el riesgo de versionar la carpeta `.venv/` (cientos de archivos) o un `.env` con credenciales. El archivo `.DS_Store` lo genera macOS en cada carpeta que abre en el Finder; ignorarlo mantiene el repositorio limpio y evita conflictos absurdos.

Atención — los datos. Decidan esto como grupo y déjenlo escrito en el README. Si el conjunto es liviano (unos pocos MB), versionarlo facilita que cualquiera reproduzca el análisis y es lo preferible para este curso. Si es pesado, ignórenlo y documenten en el README de dónde se descarga y en qué carpeta colocarlo, porque el criterio de Reproducibilidad exige que otra persona pueda ejecutar el proyecto completo.

Nota. GitHub rechaza archivos de más de 100 MB y advierte sobre los mayores a 50 MB. Si su conjunto supera ese tamaño, no intente forzarlo: documente la fuente.

Comprueba. Ejecute `git status`. ¿Aparece `.venv/` o `.DS_Store` en la lista? Si aparecen, su `.gitignore` no está funcionando: revise que esté en la raíz del proyecto y bien escrito.

PARTE VIII. Notebooks y Git: higiene para ciencia de datos

Tiempo de lectura | 8 min · Semana 1, Fase 2

Al terminar esta parte sus notebooks dejarán de generar diferencias ilegibles y conflictos constantes.

Este es el acoplamiento 4 de la Parte II, el punto que más problemas causa en los proyectos grupales y que las guías genéricas omiten.

El notebook como artefacto: dos naturalezas en tensión

Un notebook es simultáneamente un **documento** —código, resultados y narrativa integrados, en la tradición de la programación literaria— y un **archivo JSON** que Git trata como cualquier otro texto. Esa doble naturaleza es la fuente del conflicto.

A ello se suma un problema propio del formato: el notebook mantiene **estado oculto**. Las variables viven en el kernel, no en el archivo, de modo que el documento puede mostrar un resultado que ya no se obtendría al ejecutarlo de arriba abajo. Los contadores `In [7]`, `In [3]`, `In [12]` son la evidencia visible de esa desincronización.

Un archivo `.ipynb` guarda, dentro del mismo JSON, **las salidas de cada celda y los contadores de ejecución**. Eso provoca tres problemas:

- **Diffs ilegibles.** Cambiar una línea puede generar cientos de líneas de diferencia, porque también cambian las imágenes codificadas y los números de ejecución.

- **Conflictos de *merge* constantes.** Dos integrantes que solo *ejecutaron* el mismo notebook, sin editarlo, ya tienen versiones distintas.
- **Filtración de datos.** Una tabla o un gráfico impreso puede dejar información sensible dentro de un repositorio público.

8.1 Solución 1: limpiar salidas con nbstripout

Con nbstripout ya instalado, actívalo en el repositorio una sola vez:

```
nbstripout --install
```

Desde ese momento, cada `git add` de un notebook guardará la versión sin salidas. El notebook en pantalla no se altera; solo se limpia lo que va al historial.

Atención — coordinación grupal. nbstripout `--install` configura el repositorio **local de cada integrante**. Debe ejecutarlo cada persona del grupo después de clonar, o el beneficio se pierde en cuanto uno haga commit sin él.

Atención — y la evaluación. La rúbrica exige un notebook **ejecutado**, con resultados visibles. Con nbstripout activo, el `.ipynb` del repositorio no tendrá salidas. Resuélvanlo así: mantengan el notebook limpio en el repositorio y **exporten a PDF o HTML el notebook ejecutado** para adjuntarlo al informe (*File → Save and Export Notebook As*). Así cumplen ambas cosas: historial limpio y evidencia de ejecución.

8.2 Solución 2: emparejar con Jupyter

Como alternativa o complemento, puede vincular cada notebook a un archivo `.py` de texto plano, versionar el `.py` y dejar el `.ipynb` fuera de Git:

```
jupyter --set-formats ipynb,py:percent F1/notebooks/F1_Definicion.ipynb
```

Esto crea `F1_Definicion.py` sincronizado con el notebook. El `.py` se lee y se revisa mucho mejor en un Pull Request.

Recomendación. Para empezar, con nbstripout es suficiente. Cuando el trabajo grupal se intensifique en la Fase 3, añadan el emparejamiento con Jupyter.

Comprueba. Ejecute una celda de su notebook, guárdelo y corra `git diff`. Si ve las salidas de la celda en el *diff*, nbstripout no está activo. # PARTE IX. Reproducibilidad del entorno

Tiempo de lectura | 8 min · Semana 1, Fase 2

Al terminar esta parte otra persona podrá recrear su entorno de forma idéntica.

En un contexto científico, quien revise su trabajo debe poder reproducirlo. Cada vez que instale o actualice paquetes, guarde la lista exacta:

```
python -m pip freeze > requirements.txt
```

Versione el `requirements.txt`. Cualquiera podrá reconstruir el entorno con estos tres comandos, **en este orden**:

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

```
python -m pip install -r requirements.txt
```

Atención — Anaconda. Si su Python viene de Anaconda o conda, `pip freeze` puede escribir paquetes con rutas locales (`package @ file:///Users/...`) que fallan al instalar en otro equipo con «No such file or directory». Genere la lista con `python -m pip list --format=freeze > requirements.txt`, o escriba a mano solo los paquetes que su proyecto usa.

Atención — equipos mixtos. Algunos paquetes tienen versiones distintas según el sistema operativo o la arquitectura. Si un compañero no logra instalar el `requirements.txt` generado en su Mac, no lo edite a ciegas: dejen en el archivo solo las dependencias que el proyecto realmente importa, sin las transitivas, y anoten en el README la versión de Python usada.

9.1 Qué garantiza realmente un archivo de dependencias

`pip freeze` fija las versiones de los paquetes de Python, y eso resuelve una parte del problema. No resuelve las otras tres:

Fuente de variación	¿La cubre requirements.txt?
Versiones de las librerías de Python	Sí
Versión del intérprete de Python	No — decláresela en el README
Librerías del sistema y compiladores	No
Sistema operativo y arquitectura del procesador	No

Existe una gradación de mecanismos, con costo creciente: un archivo de dependencias fija versiones; un *lockfile* —el que producen herramientas como uv, Poetry o pixi— fija además el árbol completo de dependencias transitivas de forma determinista; y un contenedor fija también el sistema operativo y las librerías nativas.

Para este módulo, requirements.txt más la versión de Python declarada en el README cumple el estándar exigido. Conocer la gradación importa para saber **qué se está afirmando** cuando se afirma que un proyecto es reproducible.

9.2 Reproducibilidad no es solo el requirements.txt

Un entorno idéntico no garantiza resultados idénticos. Tres puntos adicionales que la rúbrica revisa bajo **Reproducibilidad y Validación técnica**:

Rutas relativas, siempre. `pd.read_csv("/Users/camila/Desktop/datos.csv")` funciona en un solo computador del mundo. Use `pd.read_csv("data/raw/datos.csv")` y ejecute el notebook desde la raíz del proyecto.

Semillas aleatorias. Cualquier operación con azar —muestreo, división de datos, inicialización— debe fijar la semilla, o los resultados cambiarán en cada ejecución y nadie podrá verificar los suyos:

```
import numpy as np
```

```
SEMILLA = 42
```

```
rng = np.random.default_rng(SEMILLA)
```

Ejecución limpia antes de entregar. En JupyterLab: *Kernel* → *Restart Kernel and Run All Cells*. Si el notebook falla, es que dependía del orden en que usted fue ejecutando celdas, no del orden en que están escritas. Los contadores In [7], In [3], In [12] lo delatan en la revisión.

Nota. Existen herramientas modernas (uv, pixi, Poetry) que generan un *lockfile* con versiones exactas y resuelven dependencias más rápido. Para este módulo, requirements.txt cumple perfectamente.

Comprueba. ¿Su notebook corre completo tras *Restart Kernel and Run All Cells*, en una carpeta recién clonada, sin editar ninguna ruta?

PARTE X. Un día de trabajo completo, de principio a fin

Tiempo de lectura | 8 min · Semana 1, Fase 2

Al terminar esta parte habrá visto las cinco piezas funcionando de manera conjunta en una jornada real.

Las partes anteriores explican cada herramienta por separado. Esta muestra cómo se usan juntas. Sígala completa una vez: es el ciclo que repetirá durante todo el semestre.

10.1 Abrir el proyecto

```
cd proyecto-grupoX-mcdi500      # 1. situarse en el proyecto
source .venv/bin/activate         # 2. activar el entorno -> aparece (.venv)
git pull                          # 3. traer lo que hizo el equipo
```

Atención. El orden de estos tres comandos no es casual. `git pull` **antes** de trabajar, no después: si empieza a editar sobre una versión desactualizada, el conflicto es inevitable.

10.2 Trabajar

```
python -m jupyter lab             # abre JupyterLab en el navegador
```

Dentro del notebook, la primera celda es la verificación de la Parte V. Después, el trabajo del día.

Recomendación. Guarde el notebook con frecuencia (⌘ + S). Guardar **no** es lo mismo que hacer commit: guardar deja el archivo en el disco; el commit lo registra en la historia.

10.3 Registrar el avance

Al terminar un bloque de trabajo con sentido propio —no al final del día, sino cada vez que algo queda funcionando:

```
git status                        # 1. ¿qué cambió realmente?
git add F2/notebooks/limpieza.ipynb # 2. preparar lo que corresponde
git commit -m "feat: agrega imputacion de nulos por mediana en bmi"
```

Recomendación. Prefiera `git add` archivo a `git add .` mientras aprende. Obliga a mirar qué está subiendo, que es exactamente lo que evita los accidentes.

10.4 Compartir con el equipo

```
git pull                          # otra vez: alguien pudo subir mientras trabajaba
git push
```

10.5 Cerrar el día

Si instaló alguna librería nueva durante la jornada:

```
python -m pip freeze > requirements.txt
git add requirements.txt
git commit -m "docs: actualiza dependencias del entorno"
git push
```

Y una última comprobación:

```
git status                        # debe responder: nothing to commit, working tree clean
```

Si responde otra cosa, hay trabajo sin subir. Descubrirlo ahora es mejor que descubrirlo la noche de la entrega.

10.6 El ciclo, en una línea

`cd → activate → pull → trabajar → status → add → commit → pull → push`

Comprueba. ¿Podría ejecutar este ciclo completo, de memoria, sin consultar la guía? Ese es el objetivo de la Semana 1.

PARTE XI. GitHub: cuenta y repositorio del grupo

Tiempo de lectura | 7 min · Semana 1, Fase 1

Al terminar esta parte el grupo tendrá un repositorio remoto con todos los integrantes con acceso de escritura.

11.1 Crear la cuenta

Entre a <https://github.com>, regístrese **con el mismo correo que configuró en Git**, confirme y acceda.

•

Recomendación. Use un nombre de usuario profesional. Sus repositorios pueden formar parte de su portafolio, y este proyecto es de los primeros que podrá mostrar.

11.2 Crear el repositorio remoto

En GitHub: **New repository**, asígnele el nombre acordado (proyecto-grupoX-mcdi500), elija público o privado y créelo. **No marque** la opción de añadir README ni .gitignore si ya los tienen en local: evitarán conflictos en el primer *push*.

Nota. Si lo crean privado, agreguen como colaborador al docente cuando se les indique, o el repositorio no podrá revisarse. Un enlace que el revisor no puede abrir se evalúa como componente no verificable.

11.3 Agregar a los integrantes del grupo

Una sola persona crea el repositorio. Luego, en GitHub: *Settings* → *Collaborators* → *Add people*, e invite a los demás por su nombre de usuario. Cada uno recibe una invitación por correo que debe aceptar.

Desde ese momento, todos clonan el mismo repositorio:

```
git clone https://github.com/USUARIO/proyecto-grupoX-mcdi500.git
cd proyecto-grupoX-mcdi500
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
nbstripout --install
python -m ipykernel install --user --name grupoX_mcdi500 \
    --display-name "Python (grupoX-mcdi500)"
```

Fíjese en que este bloque reconstruye **las cinco piezas** en el computador de cada integrante: el código llega con el `clone`, el entorno con `venv`, las librerías con `requirements.txt`, la higiene de notebooks con `nbstripout` y el acoplamiento con Jupyter mediante el kernel.

Rúbrica. Esta es la única forma de que el historial muestre commits de cada integrante. La alternativa habitual —que una persona reciba los archivos por mensajería y los suba— produce un repositorio con un solo autor, que en el criterio de Repositorio GitHub no alcanza el nivel Excelente por más completo que esté el contenido.

Comprueba. ¿Cada integrante puede ejecutar `git push` sin error de permisos? Pruébenlo en la primera semana, no la noche de la entrega.

PARTE XII. Conectar Git local con GitHub

Tiempo de lectura | 5 min · Semana 1, Fase 1

Si usted **clonó** el repositorio (Parte 11.3), el remoto ya está configurado y puede saltar a 12.3. Si empezó en local, siga desde aquí.

12.1 Agregar el remoto

```
git remote add origin https://github.com/USUARIO/proyecto-grupoX-mcdi500.git
```

origin es el nombre convencional del repositorio remoto principal.

12.2 Verificar el remoto

```
git remote -v
```

12.3 Autenticarse con GitHub CLI

```
brew install gh
gh auth login
```

Responda: **GitHub.com** → protocolo **HTTPS** → autenticar Git con sus credenciales **Yes** → **Login with a web browser**. Se abrirá el navegador; inicie sesión, autorice y vuelva a la Terminal. Verifique con:

```
gh auth status
```

Nota. Si no tiene Homebrew, puede descargar la CLI desde <https://cli.github.com>, o instalar Homebrew siguiendo las instrucciones de <https://brew.sh>.

12.4 Subir el proyecto por primera vez

```
git push -u origin main
```

Nota. -u deja enlazada la rama local con la remota. Después basta con `git push`.

PARTE XIII. Flujo de trabajo en equipo

Tiempo de lectura | 6 min · Semana 1, Fase 2

Al terminar esta parte sabrá el ciclo diario y por qué `git pull` va antes que `git push`.

13.1 El ciclo individual

```
git status           # revisar qué cambió
git add .            # preparar
git commit -m "Describe el avance" # registrar
git push             # subir
```

13.2 El ciclo en grupo

Cuando varias personas trabajan sobre el mismo repositorio hay un paso más, y omitirlo es la causa de la mayoría de los problemas:

```
git pull             # ANTES de empezar: traer lo que hicieron los demás
# ... trabajar ...
git status
git add .
git commit -m "Describe el avance"
git pull             # otra vez: alguien pudo subir algo mientras trabajaba
git push
```

13.3 pull frente a fetch

Conviene entender la diferencia, porque explica qué ocurre cuando el `pull` produce un conflicto:

- **git fetch** trae los commits del remoto pero **no** los mezcla con su trabajo. Su carpeta no cambia.
- **git merge** fusiona esos commits con los suyos.
- **git pull** hace las dos cosas de una vez: `fetch` + `merge`.

Por eso un conflicto aparece durante el `pull`: es el `merge` interno el que no sabe qué versión conservar.

Atención. Si al hacer `git push` aparece `rejected ... non-fast-forward`, significa que el remoto tiene commits que usted no tiene. **No fuerce el push con `--force`**: sobrescribiría el trabajo de sus compañeros y sería irrecuperable. Haga `git pull`, resuelva lo que haga falta (Parte XVI) y vuelva a subir.

Recomendación — acuerdo de grupo. Repártanse el trabajo **por archivo, no por línea**. Dos personas editando el mismo notebook a la misma hora garantiza conflictos; dos personas trabajando en `F2/notebooks/limpieza.ipynb` y `F2/src/transformaciones.py` no chocan nunca. Definan esto en la reunión de planificación de cada fase.

Comprueba. ¿Sabe qué hace `git pull` que no hace `git fetch`?

PARTE XIV. Ramas

Tiempo de lectura | 5 min · Semana 2, Fase 3

Al terminar esta parte podrá trabajar en una funcionalidad sin arriesgar la versión estable.

14.1 ¿Qué es una rama?

Una rama es una **línea de trabajo independiente** dentro del mismo proyecto. Sirve para desarrollar, corregir o experimentar sin afectar la versión estable. `main` representa esa versión estable: lo que está en `main` debería funcionar siempre.

En el proyecto del curso la utilidad es directa: si está probando tres formas de imputar valores faltantes, cada una puede vivir en su rama y solo la elegida se integra a `main`.

14.2 Crear y cambiar de rama

```
git switch -c f2-limpieza-datos    # crear y cambiar en un paso
git branch                       # ver ramas (la actual con *)
git switch main                   # volver a la principal
```

Nota. Verá también `git checkout` en muchos tutoriales: hace lo mismo, pero `git switch` y `git restore` son la forma moderna y más clara, porque separan «cambiar de rama» de «descartar cambios».

Recomendación. Nombre las ramas por lo que hacen y por la fase: `f2-limpieza-datos`, `f3-rekursividad`, `f3-clase-dataset`. Evite `ramal`, `prueba`, `camila`.

14.3 Trabajar y guardar en la rama

```
git add .
git commit -m "feat: agrega imputacion de valores faltantes por mediana"
git push -u origin f2-limpieza-datos
```

PARTE XV. Colaboración con Pull Requests

Tiempo de lectura | 6 min · Semana 2, Fase 3

Al terminar esta parte sabrá integrar cambios mediante revisión, no a ciegas.

En GitHub, la forma correcta de integrar y revisar cambios es el **Pull Request (PR)**. Es el mecanismo que usará cuando trabaje en grupo y el que replica lo que ocurre en cualquier equipo de datos profesional.

```
# 1. Crear y trabajar en una rama
git switch -c f3-medicion-complejidad
# ... editar archivos ...
git add .
git commit -m "feat: mide complejidad temporal de la búsqueda recursiva"
```

```
# 2. Subir la rama al remoto
git push -u origin f3-medicion-complejidad
```

3. En GitHub aparecerá un botón para abrir un Pull Request desde esa rama hacia `main`. Ábralo, describa el cambio y solicite revisión a un compañero.
4. El revisor comenta línea por línea. Cuando esté conforme, se fusiona el PR desde la interfaz de GitHub.
5. Actualice su `main` local:

```
git switch main
git pull
git branch -d f3-medicion-complejidad
```

Recomendación — cómo revisar el PR de un compañero. Revisar no es aprobar por cortesía. Preguntas útiles: ¿el código hace lo que dice el mensaje del commit? ¿las transformaciones están justificadas o simplemente aplicadas? ¿corre si lo ejecuto desde cero?

¿los nombres de variables se entienden sin preguntar? Un comentario concreto en una línea vale más que un «se ve bien».

Rúbrica. El PR deja registro de la discusión y de la revisión, no solo del código. Es evidencia directa para el criterio de **Documentación del proceso**, que pide justificar decisiones técnicas indicando motivo e impacto.

15.1 Fusión local (alternativa simple)

Para trabajo individual o cambios menores:

```
git switch main
git merge f3-medicion-complejidad
git push
git branch -d f3-medicion-complejidad
```

PARTE XVI. Conflictos: qué son y cómo resolverlos

Tiempo de lectura | 6 min · Semana 2, Fase 3

Al terminar esta parte un conflicto dejará de ser una emergencia.

Un conflicto ocurre cuando **dos personas modifican la misma parte del mismo archivo** y Git no puede decidir cuál versión conservar. No es un error ni un castigo: es Git pidiendo que se tome una decisión que solo el equipo puede tomar.

16.1 Cómo se ve

```
<<<<<<< HEAD
df = df.dropna(subset=["edad"])
=====
df["edad"] = df["edad"].fillna(df["edad"].median())
>>>>>>> f2-limpieza-datos
Arriba, su versión. Abajo, la que viene de la otra rama.
```

16.2 Cómo se resuelve

1. Abra el archivo y **converse con su compañero** si el conflicto es de criterio técnico, como en el ejemplo: eliminar filas o imputar la mediana no son equivalentes, y la decisión debe quedar justificada en el informe.
2. Edite el archivo dejando el contenido final correcto y **borre las tres líneas de marcas** (<<<<<<<, =====, >>>>>>>).
3. Confirme la resolución:

```
git add archivo_en_conflicto.py
git commit -m "fix: resuelve conflicto en estrategia de imputacion"
```

Si desea abortar y volver al estado anterior: `git merge --abort`.

16.3 Conflictos en notebooks

Un conflicto dentro de un `.ipynb` es prácticamente ilegible, porque el archivo es JSON con salidas incrustadas. Tres formas de tratarlo, en orden:

- **Prevenir:** `nbstripout` activo en todos los equipos (Parte VIII) reduce drásticamente la frecuencia.
- **Repartir:** que dos personas no editen el mismo notebook simultáneamente.
- **Resolver:** si ya ocurrió, lo más práctico es quedarse con una versión completa —`git checkout --ours archivo.ipynb` o `--theirs`— y reincorporar a mano las celdas de la otra, en vez de intentar editar el JSON.

Comprueba. Provoquen un conflicto a propósito en un archivo de prueba y resuélvanlo juntos. Es mucho mejor aprender esto en la Semana 1 que la noche antes de la Sumativa de la Fase 3.

PARTE XVII. EL README técnico

Tiempo de lectura | 6 min · Semanas 1 y 3, Fases 1 y 4

Al terminar esta parte tendrá un README que cumple el criterio de la rúbrica.

El README es lo primero que ve quien abre el repositorio, y en este curso es **evidencia evaluada**. Un README que dice «Proyecto de MCDI500» no cumple. Uno que permite a un tercero ejecutar el proyecto sin preguntar nada, sí.

17.1 Plantilla

Proyecto Grupo X – MCDI500

Breve descripción de la problemática abordada y del objetivo del análisis (2 o 3 frases).

Integrantes

- Nombre Apellido (@usuario-github)
- Nombre Apellido (@usuario-github)

Datos

Fuente del conjunto, licencia o condiciones de uso, número de registros y variables. Si el conjunto no está versionado: dónde descargarlo y en qué carpeta colocarlo.

Estructura del repositorio

- F1/ Definición del problema y entorno reproducible
- F2/ Obtención, limpieza y transformación de datos
- F3/ Núcleo algorítmico: programación estructurada, recursiva y P00
- F4/ Análisis, visualización y comunicación de resultados

Requisitos y ejecución

Python 3.13

```
python3 -m venv .venv
source .venv/bin/activate      # macOS y Linux
# .venv\Scripts\activate      # Windows
python -m pip install -r requirements.txt
```

Ejecutar los notebooks en orden desde la raíz del proyecto.

Convención de commits

Prefijos usados: docs, data, feat, fix. Ver Parte XVIII de la guía.

Decisiones técnicas

Registro breve de las decisiones relevantes y su motivo.

Ejemplo: "Se imputa la edad con la mediana en lugar de eliminar registros, porque los NA representan el 12% de la muestra y su eliminación sesgaría la distribución por región."

Recomendación. Documenten la activación del entorno para macOS y para Windows, como en la plantilla. Es lo que evita que el integrante con el otro sistema quede bloqueado en el peor momento.

Recomendación. Actualice el README al cerrar cada fase, no al final del curso. La sección de decisiones técnicas escrita en el momento es la que después evita reconstruir de memoria la justificación para el informe.

PARTE XVIII. Mensajes de commit

Tiempo de lectura | 4 min · Semana 1, Fase 2

Al terminar esta parte sus commits comunicarán algo a quien lea el historial, incluido usted mismo la próxima semana.

18.1 La convención del curso

Use un prefijo que indique el tipo de cambio, seguido de una descripción en presente:

Prefijo	Se usa para	Ejemplo
feat	Funcionalidad o análisis nuevo	feat: agrega funcion de normalizacion de variables numericas
fix	Corrección de un error	fix: corrige indice desalineado tras el merge de tablas
data	Cambios en datos o su procesamiento	data: incorpora dataset de accidentes 2024 en raw
docs	Documentación, README, informe	docs: documenta criterios de limpieza en el README

18.2 Qué distingue un buen mensaje

Evite	Prefiera
cambios	feat: agrega deteccion de duplicados por RUT
avance	data: elimina 47 registros duplicados del dataset base
arreglado	fix: corrige lectura de fechas con formato dd-mm-yyyy
subo notebook	docs: documenta decisiones de imputacion en notebook F2

La prueba es simple: **si alguien lee solo su historial, ¿entiende cómo evolucionó el proyecto?** Ese es literalmente el ejercicio que hace quien evalúa.

Recomendación. Commits pequeños y frecuentes, no uno enorme al final del día. Un commit debería corresponder a un cambio con sentido propio: «agrega la limpieza de nulos» es un commit; «trabajo del martes» son ocho commits mezclados.

Atención. Evite tildes y caracteres especiales en los mensajes de commit. Aunque macOS los maneja sin problemas, el equipo puede incluir computadores con Windows donde, según la configuración de la terminal, aparecen corruptos en el historial compartido.

PARTE XIX. Diagnóstico de errores por acoplamiento

Tiempo de lectura | consulta puntual

Los errores se ordenan aquí según **qué acoplamiento de la Parte II se rompió**. Identificar la pieza es la mitad de la solución.

19.1 Acoplamiento 1 — El entorno virtual no está activo

command not found: python En macOS el comando python a secas no existe mientras no haya un entorno virtual activo. Use python3, o active el entorno con `source .venv/bin/activate`.

error: externally-managed-environment Intentó instalar paquetes en el Python del sistema, que macOS protege. La solución no es forzar la instalación:

```
source .venv/bin/activate
python -m pip install nombre_paquete
```

Instaló un paquete y «desapareció» Casi siempre el entorno no estaba activo al instalar, o abrió una terminal nueva y olvidó activarlo. Verifique que ve (.venv) al inicio de la línea.

19.2 Acoplamiento 2 — El kernel no corresponde al entorno

ModuleNotFoundError con un paquete que sí instaló El notebook está usando el kernel «Python 3» genérico en lugar del kernel del proyecto. Ejecute en una celda `import sys; print(sys.executable)`: si la ruta no contiene .venv, cambie el kernel con *Kernel* → *Change Kernel*.

El kernel del proyecto no aparece en la lista No se registró, o se registró desde otro entorno. Con el entorno activo:

```
python -m ipykernel install --user --name grupoX_mcdi500 \
    --display-name "Python (grupoX-mcdi500)"
```

FileNotFoundError con una ruta relativa correcta El notebook se está ejecutando desde otra carpeta. Compruébelo con `from pathlib import Path; print(Path.cwd())` y abra JupyterLab desde la raíz del proyecto.

19.3 Acoplamiento 3 — El entorno no es reproducible

El compañero no puede instalar el requirements.txt Vea las advertencias sobre Anaconda y equipos mixtos en la Parte IX.

El notebook falla tras Restart & Run All aunque «funcionaba» Dependía del orden en que usted ejecutó las celdas. Reordene el código para que el flujo de arriba abajo sea correcto.

19.4 Acoplamiento 4 — Notebooks y Git

Un cambio mínimo genera cientos de líneas de diferencia nbstripout no está activo en ese repositorio. Ejecute `nbstripout --install`.

Conflicto ilegible dentro de un .ipynb Vea la Parte XVI, sección 16.3.

19.5 Acoplamiento 5 — Git y GitHub

fatal: unable to auto-detect email address Git no tiene configurado nombre o correo:

```
git config --global user.name "Nombre"
git config --global user.email "correo@ejemplo.com"
```

src refspec main does not match any Todavía no existe ningún commit en main:

```
git add .
git commit -m "docs: primer commit"
git push -u origin main
remote origin already exists
git remote set-url origin NUEVA_URL
```

Updates were rejected because the remote contains work that you do not have locally El remoto tiene commits que usted no tiene. **No use --force**:

```
git pull
# resolver conflictos si aparecen
git push
```

this exceeds GitHub's file size limit of 100.00 MB Intentó subir un archivo demasiado grande, normalmente un conjunto de datos. Agréguelo al .gitignore y documente en el README dónde obtenerlo. Si ya lo registró en un commit, consulte en el Foro antes de reescribir el historial por su cuenta.

Subió por error .venv/, .DS_Store o un archivo con credenciales

```
git rm -r --cached .venv
git rm --cached .DS_Store
git commit -m "fix: deja de versionar archivos que no corresponden"
```

Si eran credenciales reales, además **cámbielas**: siguen visibles en el historial.

19.6 Errores del sistema

xcrun: error: invalid active developer path Aparece tras actualizar macOS: las Command Line Tools quedaron desvinculadas. Reinstálelas con `xcode-select --install`.

zsh: command not found: brew Homebrew no está instalado o no está en el PATH. Instálelo desde <https://brew.sh> y reinicie la Terminal.

Nota. Si el error que aparece no está en esta lista, publíquelo en el Foro de Consultas indicando el comando ejecutado, lo que esperaba y el mensaje exacto.

PARTE XX. Integridad académica en el repositorio

Tiempo de lectura | 3 min

El historial de Git es un **registro de autoría**. Eso tiene tres implicancias directas.

El trabajo debe ser propio. Las guías de apoyo del curso entregan esqueletos con marcas TODO para orientar, no soluciones para copiar; el plagio se sanciona con nota mínima. Un repositorio cuyo primer commit contiene el proyecto completo y terminado es, como mínimo, una señal de que el proceso no ocurrió como se declara.

El uso de asistentes de IA debe ser transparente. Si utilizó un asistente para generar o depurar código, declárelo según los lineamientos de uso ético publicados en el curso y —esto es lo importante— asegúrese de **entender y poder explicar cada línea que entrega**. La evaluación distingue entre obtener un resultado correcto y comprender por qué lo es.

La contribución individual es visible. El historial muestra quién hizo qué y cuándo. Esto protege a quien trabaja, y por eso el criterio de la rúbrica lo exige.

PARTE XXI. Checklist antes de cada entrega

Tiempo de lectura | 4 min · antes de cada evaluación

Revise esta lista con el repositorio abierto, no de memoria.

Entorno

- El entorno virtual se activa sin errores y `which python` apunta dentro de `.venv`.
- El kernel del proyecto aparece en JupyterLab y los notebooks lo tienen seleccionado.
- `requirements.txt` regenerado después de la última instalación de paquetes.

Repositorio

- La estructura de carpetas corresponde a las fases entregadas (F1, F2, ...).
- `.gitignore` presente y funcionando: `.venv/` y `.DS_Store` no aparecen en GitHub.
- README actualizado, con estructura, dependencias, instrucciones de ejecución para macOS y Windows, y decisiones técnicas de la fase.
- Todo lo local está subido: `git status` responde *nothing to commit, working tree clean*.

Historial

- Hay commits de **cada** integrante del grupo.
- Los mensajes describen el cambio y usan los prefijos de la convención.
- Los commits están repartidos en el tiempo, no todos en las últimas horas.

Notebooks

- Corren completos tras *Restart Kernel and Run All Cells*.
- Usan rutas relativas.
- Fijan semilla aleatoria donde hay azar.

- Las celdas narrativas explican **por qué** se hizo cada paso, no solo qué se hizo.
- Se adjunta la versión ejecutada (PDF o HTML) si el repositorio guarda los notebooks limpios.

Acceso

- El enlace del repositorio está en el informe.
- Si es privado, el docente tiene acceso. Verifíquenlo abriendo el enlace desde una sesión distinta.

Atención. Un componente que el revisor no puede verificar no se asume cumplido. Un repositorio privado sin acceso equivale, para efectos de la evaluación, a un repositorio ausente.

PARTE XXII. Secuencia consolidada desde cero

Tiempo de lectura | consulta puntual

22.1 Del inicio a la publicación

```
# Configuración (una sola vez por equipo)
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
git config --global init.defaultBranch main

# Crear proyecto y entorno
mkdir proyecto-grupoX-mcdi500 && cd proyecto-grupoX-mcdi500
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install jupyterlab numpy pandas matplotlib seaborn \
    scikit-learn jupyter nbstripout

# Registrar el kernel del proyecto
python -m ipykernel install --user --name grupoX_mcdi500 \
    --display-name "Python (grupoX-mcdi500)"

# Inicializar Git y limpiar notebooks
git init
nbstripout --install

# Crear archivos (incluido .gitignore CON contenido)
touch README.md
mkdir -p F1/data/raw F1/data/processed F1/notebooks F1/src F1/docs
# ... escribir el .gitignore ...

# Guardar dependencias y primer commit
python -m pip freeze > requirements.txt
git add .
git commit -m "docs: crea estructura inicial del proyecto"

# Conectar con GitHub y subir
git remote add origin https://github.com/USUARIO/proyecto-grupoX-mcdi500.git
git push -u origin main
```

22.2 Incorporarse a un repositorio existente (resto del grupo)

```
git clone https://github.com/USUARIO/proyecto-grupoX-mcdi500.git
cd proyecto-grupoX-mcdi500
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
nbstripout --install
python -m ipykernel install --user --name grupoX_mcdi500 \
    --display-name "Python (grupoX-mcdi500)"
```

22.3 Desarrollo con ramas y Pull Request

```
git pull
git switch -c f2-limpieza-datos
# ... trabajar ...
git add .
git commit -m "feat: agrega pipeline de limpieza de valores faltantes"
git push -u origin f2-limpieza-datos
# Abrir el Pull Request en GitHub, revisar y fusionar
git switch main
git pull
git branch -d f2-limpieza-datos
```

Glosario

Término	Significado
Repositorio	Carpeta del proyecto con su historial de Git.
Commit	Registro guardado de un conjunto de cambios, con autor, fecha y mensaje.
Staging	Zona intermedia donde se preparan los cambios del próximo commit.
Rama (<i>branch</i>)	Línea de trabajo independiente dentro del mismo repositorio.
main	Rama principal; la versión estable del proyecto.
origin	Nombre convencional del repositorio remoto en GitHub.
Clonar	Descargar una copia completa de un repositorio remoto, con su historial.
Push / Pull	Subir los commits propios al remoto / traer los commits del remoto.
Fetch	Traer los commits del remoto sin fusionarlos con el trabajo local.
Pull Request (PR)	Propuesta de integrar una rama en otra, abierta a revisión y discusión.
Merge	Fusión de dos ramas.
Conflicto	Situación en que dos versiones modifican la misma parte de un archivo.
Kernel	Intérprete de Python que ejecuta un notebook.
Entorno virtual	Instalación de Python aislada, propia de un proyecto.
Homebrew	Gestor de paquetes de macOS; instala herramientas como git o gh.
Command Line Tools	Herramientas de desarrollo de Apple, necesarias para compilar librerías científicas.

Autoevaluación

Tiempo de lectura | 8 min

Las primeras preguntas verifican el manejo operativo; las últimas, la comprensión del fundamento. Si puede responderlas sin consultar el documento, domina lo que el módulo espera.

	Pregunta	Volver a
1	¿Qué hace cada una de las cinco piezas del ecosistema, y qué no hace?	Parte II
2	¿Por qué el kernel del notebook debe ser el del entorno del proyecto?	Partes II y IV
3	¿Qué distingue <code>git add</code> de <code>git commit</code> , y por qué existe un paso intermedio?	Parte VI
4	¿Por qué en macOS se usa <code>python3</code> antes de activar el entorno y <code>python</code> después?	Partes III y IV
5	¿Por qué <code>git pull</code> debe preceder a <code>git push</code> en trabajo colaborativo?	Parte XIII
6	¿Qué problema específico del formato <code>.ipynb</code> resuelve <code>nbstripout</code> ?	Parte VIII
7	¿Por qué versionar el <code>.ipynb</code> no basta para que otra persona reproduzca su análisis?	Parte IX
8	Su compañero hizo <i>push</i> y el suyo es rechazado. ¿Qué hace, y qué no debe hacer nunca? ¿Por qué?	Partes VI, XIII y XIX
9	¿Por qué alterar un commit antiguo invalida los identificadores de todos los posteriores?	Parte VI
10	¿Qué fuentes de variación no cubre un <code>requirements.txt</code> , y qué mecanismo las cubriría?	Parte IX
11	¿Qué diferencia hay entre repetibilidad y reproducibilidad, y cuál exige la rúbrica?	Parte I
12	¿Qué evidencia del trabajo colaborativo queda en el historial que no queda en el informe?	Partes XI y XX

Estas preguntas son también un punto de partida para las intervenciones en el Foro de reflexión de cada fase, donde se espera que fundamente decisiones técnicas con ejemplos de su propia práctica.

Conclusión

Git y GitHub no son herramientas auxiliares del análisis: son la infraestructura que permite sostener una afirmación computacional. En un contexto donde el resultado se presenta como evidencia, poder demostrar **cómo se llegó a él, quién intervino y bajo qué condiciones se obtuvo** no es un requisito administrativo del curso, sino la condición mínima de validez del trabajo.

Ninguna de las cinco piezas cumple esa función por separado. Python ejecuta, el entorno virtual aísla, JupyterLab documenta, Git registra y GitHub distribuye y somete a revisión. Lo que este módulo evalúa no es el conocimiento aislado de cada una, sino la **capacidad de**

•

articularlas: un notebook que se ejecuta en el entorno declarado, versionado con un historial legible, en un repositorio que un tercero puede clonar y ejecutar sin intervención.

Esa articulación tiene un nombre en la literatura de investigación computacional, y es el criterio con que se juzga hoy un trabajo de datos: reproducibilidad.

La idea central. Ciclo de trabajo: `cd` → **activar el entorno** → `git pull` → trabajar → `git status` → `git add` → `git commit` → `git push`. Cuando el proyecto crece: crear rama → trabajar → *commitear* → subir rama → abrir Pull Request → revisar → fusionar.

Referencias

Verifique los datos de cada fuente antes de citarla en sus informes.

Chacon, S., & Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. <https://git-scm.com/book>

Peng, R. D. (2011). Reproducible research in computational science. *Science*, 334(6060), 1226–1227.

Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10), e1003285.

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510.

Documentación técnica oficial

Git. (s.f.). *Git reference manual*. <https://git-scm.com/docs>

GitHub. (s.f.). *GitHub Docs*. <https://docs.github.com>

Project Jupyter. (s.f.). *JupyterLab documentation*. <https://jupyterlab.readthedocs.io>

Python Software Foundation. (s.f.). *venv — Creation of virtual environments*. <https://docs.python.org/3/library/venv.html>

Documento técnico de apoyo · MCDI500 — Programación para la Ciencia de Datos · Magíster en Ciencia de Datos e Inteligencia Artificial · Facultad de Ingeniería · Universidad Andrés Bello · 2026